Data and Information Systems Management

METI

Final Project [EN]

Authors:

Afonso Gomes (98921) afonso.a.m.gomes@tecnico.ulisboa.pt
Tiago Videira (116069) tiago.g.videira@tecnico.ulisboa.pt

Group 25

2025/2026 – 2nd Semester, P3

Contents

1 Task 1: Database Creation and Data Insertion **2**
1.1 Table Creation . . . 2
1.2 Data Import . . . 2
1.3 Short Tables Creation . . . 3
1.4 Verification . . . 3
1.5 Discussion . . . 4

2 Task 2: Data Storage (Partitioning) **5**
2.1 Partition Function and Scheme . . . 5
2.2 Partitioned Table . . . 5
2.3 Data Loading . . . 5
2.4 Partition Verification . . . 6
2.5 Discussion . . . 7

3 Task 3: Indexing and Query Optimization **8**
3.1 Query . . . 8
3.2 Index Creation . . . 8
3.3 Execution Plans . . . 9
3.4 Cost Comparison . . . 9
3.5 Discussion . . . 10

4 Task 4: Join and Group By Analysis **11**
4.1 Join And Group Algorithms . . . 11
4.2 Execution Plans . . . 13
4.3 Cost Comparison . . . 15
4.4 Discussion . . . 15

5 Task 5: Query Tuning **16**
5.1 Optimized Query . . . 16
5.2 Execution Plans . . . 16
5.3 Discussion . . . 17

6 Task 6: Transactions and Isolation Levels **18**
6.1 Isolation Level Uncommitted . . . 18
6.2 Isolation Level Read Committed . . . 19
6.3 Isolation Level Serializable . . . 20
6.4 Discussion . . . 21

7 Conclusion **22**

8 Project Repository **22**
8.1 Repository Structure . . . 22

1 Task 1: Database Creation and Data Insertion

The first task consisted of importing and structuring the datasets provided by Gov.UK for the years 2024 and 2025.

1.1 Table Creation

```sql
CREATE TABLE PricePaidData2024 (
    TransactionUniqueIdentifier VARCHAR(50) NOT NULL,
    Price BIGINT NULL,
    DateOfTransfer SMALLDATETIME NULL,
    Postcode VARCHAR(8) NULL,
    PropertyType CHAR(1) NULL,
    Old_New CHAR(1) NULL,
    Duration CHAR(1) NULL,
    PAON VARCHAR(100) NULL,
    SAON VARCHAR(50) NULL,
    Street VARCHAR(50) NULL,
    Locality VARCHAR(50) NULL,
    Town VARCHAR(50) NULL,
    District VARCHAR(50) NULL,
    County VARCHAR(50) NULL,
    PPDCategoryType CHAR(1) NULL,
    RecordStatus VARCHAR(5) NULL,

  CONSTRAINT PK_PricePaidData2024
    PRIMARY KEY (TransactionUniqueIdentifier)
);
```

Listing 1: Creation of table PricePaidData2025

1.2 Data Import

```sql
BULK INSERT PricePaidData2024
FROM 'C:\Users\Administrator\Downloads\pp-2024.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 1,
    FIELDQUOTE = '"',
  ROWTERMINATOR = '0x0a',
    TABLOCK
);
```

Listing 2: Data import query

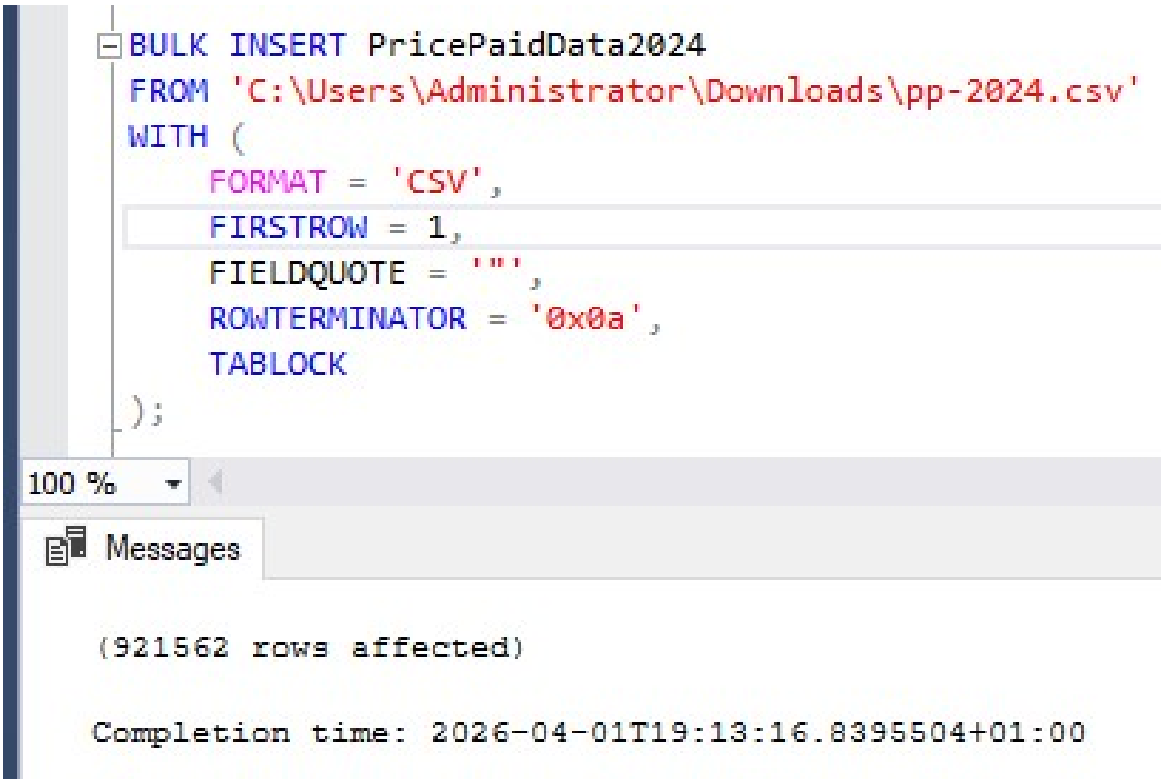


Figure 1: Successful import of CSV data

1.3 Short Tables Creation

```sql
SELECT
    TransactionUniqueIdentifier, Price, DateOfTransfer, Postcode,
    PropertyType, Old_New, PAON, Street, Locality, Town, District,
       County
INTO ShortPricePaidData2025
FROM PricePaidData2025;

SELECT
    TransactionUniqueIdentifier, Price, DateOfTransfer, Postcode,
    PropertyType, Old_New, PAON, Street, Locality, Town, District,
       County
INTO ShortPricePaidData2024
FROM PricePaidData2024;
```

Listing 3: Creation of Short tables

1.4 Verification

```sql
SELECT COUNT(*) AS PricePaidData2025 FROM PricePaidData2025;
SELECT COUNT(*) AS ShortPricePaidData2025 FROM ShortPricePaidData2025;
SELECT COUNT(*) AS PricePaidData2024 FROM PricePaidData2024;
```

```sql
SELECT COUNT(*) AS ShortPricePaidData2024 FROM ShortPricePaidData2024;
```

Listing 4: Verification of record counts

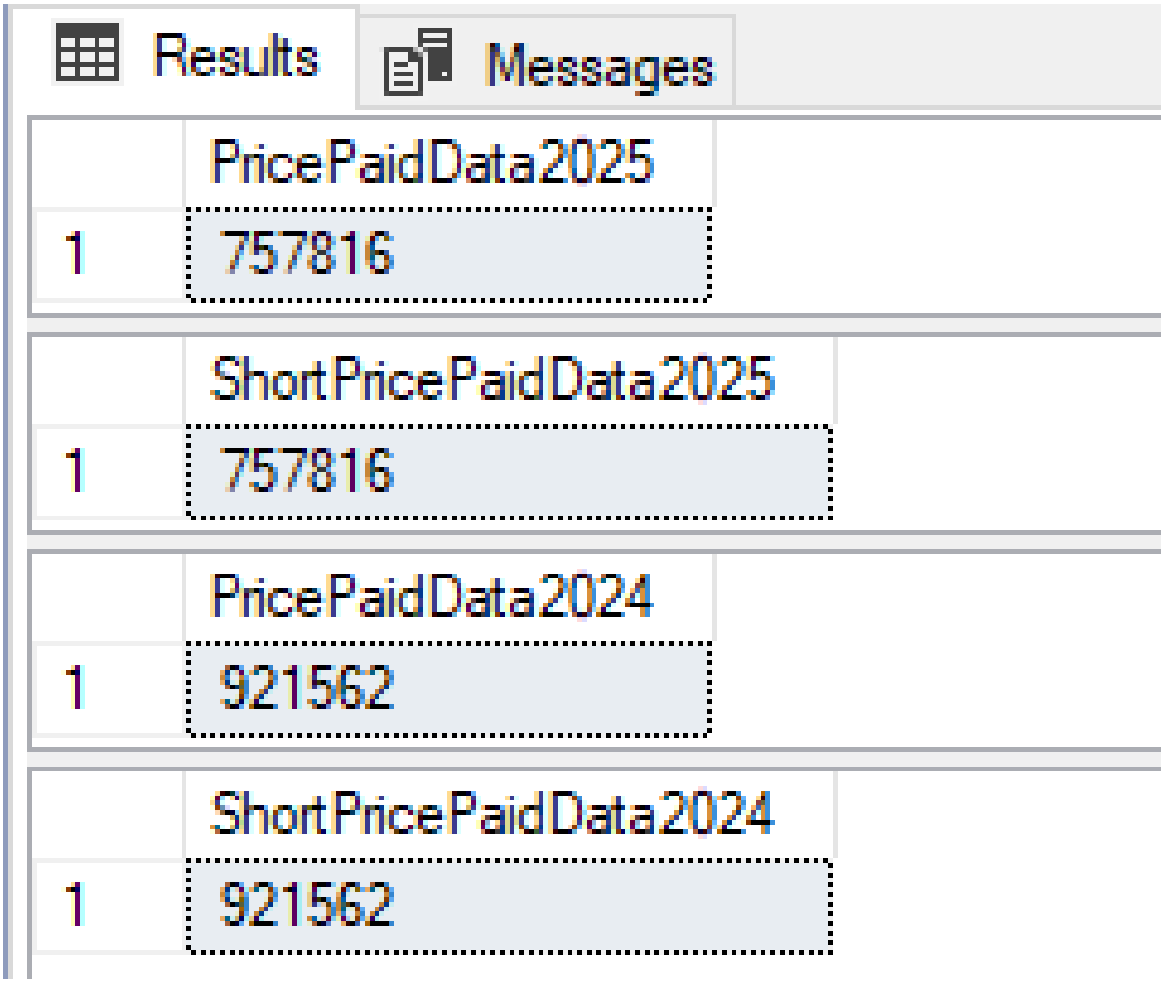


Figure 2: Verification of tuple counts

1.5 Discussion

The process of explicitly defining the schema before importing the data ensures better control over data integrity and storage efficiency. By selecting appropriate data types (e.g., `SMALLDATETIME` for dates and `VARCHAR` for textual attributes), we reduce storage overhead while preserving necessary information.

Additionally, defining the `TransactionUniqueIdentifier` as the primary key guarantees uniqueness and prevents duplicate entries, which is critical for maintaining data consistency.

The creation of the `ShortPricePaidData` tables simplifies subsequent analytical queries by removing unnecessary attributes. This reduces the size of the dataset being processed and improves query performance, especially in aggregation and filtering operations.

Finally, the verification step confirms that the transformation process did not introduce inconsistencies or data loss, ensuring reliability for further analysis.

2 Task 2: Data Storage (Partitioning)

2.1 Partition Function and Scheme

Firstly we use the NTILE function to understand around where the partitions should be done. Then we create the partitions where we separate the counties by their starting letters.

```sql
SELECT DISTINCT county,
       NTILE(5) OVER (ORDER BY county) AS partition_group
FROM (
    SELECT county
    FROM ShortPricePaidData2025
) groups
ORDER BY county;

CREATE PARTITION FUNCTION pf_County (VARCHAR(50))
AS RANGE LEFT FOR VALUES (
    'D', 'H', 'M', 'S'
);

CREATE PARTITION SCHEME ps_County
AS PARTITION pf_County
ALL TO ([PRIMARY]);
```

Listing 5: Creation of partition function and scheme

2.2 Partitioned Table

```sql
CREATE TABLE PartitionedPricePaidData2025 (
    TransactionUniqueIdentifier VARCHAR(50),
    Price BIGINT,
    DateOfTransfer SMALLDATETIME,
    Postcode VARCHAR(8),
    PropertyType CHAR(1),
    Old_New CHAR(1),
    PAON VARCHAR(100),
    Street VARCHAR(50),
    Locality VARCHAR(50),
    Town VARCHAR(50),
    District VARCHAR(50),
    County VARCHAR(50)
) ON ps_County(County);
```

Listing 6: Creation of partitioned table

2.3 Data Loading

```sql
INSERT INTO PartitionedPricePaidData2025
SELECT * FROM ShortPricePaidData2025;
```

Listing 7: Insert data into partitioned table

2.4 Partition Verification

```sql
SELECT
    $PARTITION.pf_County(County) AS Partition_Number,
    COUNT(*) AS Number_of_Records
FROM PartitionedPricePaidData2025
GROUP BY $PARTITION.pf_County(County)
ORDER BY Partition_Number;
```

Listing 8: Query to verify partition distribution

We can see here that, with the way we separated the partitions, 3 of them have a very similar number of rows; however, 2 of them have a considerable, although tolerable, amount more than the other 3 .

Results Messages

	partition_number	total_rows
1	1	108920
2	2	198174
3	3	109535
4	4	109335
5	5	231852

Figure 3: Partition distribution

2.5 Discussion

The chosen partitioning strategy is based on the `County` attribute, which is frequently used in queries. By organizing data according to this attribute, we enable SQL Server to apply partition elimination.

Partition elimination is a key optimization mechanism in which only the relevant partitions are accessed during query execution. This significantly reduces the number of rows scanned, leading to improved performance.

Although all partitions are mapped to the same filegroup, the logical separation of data still provides performance benefits. In more advanced scenarios, different partitions could be placed in separate filegroups to further optimize I/O operations.

The distribution of records across partitions also highlights how data is spread geographically, which can influence query performance depending on data skew.

3 Task 3: Indexing and Query Optimization

3.1 Query

```sql
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London'
AND Old_New = 'Y'
GROUP BY locality
ORDER BY average_new_property_price DESC;
```

Listing 9: Query under analysis

3.2 Index Creation

The first index we created was on `town` and `old_new` because these were the attributes the query uses to filter.

```sql
--first index
CREATE INDEX first_idx
ON ShortPricePaidData2025(town, old_new)
```

The second created index was the same as the previous, but we put `price` and `locality` on the leaves. This also corresponds to the suggestion of the SQL SERVER.

```sql
--second index
CREATE INDEX second_idx
ON ShortPricePaidData2025(town, old_new)
INCLUDE(price, locality);
```

The third and last relevant index created was with `town`, `old_new` and `locality` on the index and only `price` on the leaves, the reason is because the locality is used in a `GROUP BY`, so it's worthwhile to use it on the index. It's also important to note that locality is last on the index since `town` and `old_new` are more important due to filtering, we also tried to switch these two attributes with each other to understand the most relevant order on the index.

```sql
--third index
CREATE INDEX best_idx
ON ShortPricePaidData2025(old_new, town, locality)
INCLUDE(price);
```

3.3 Execution Plans

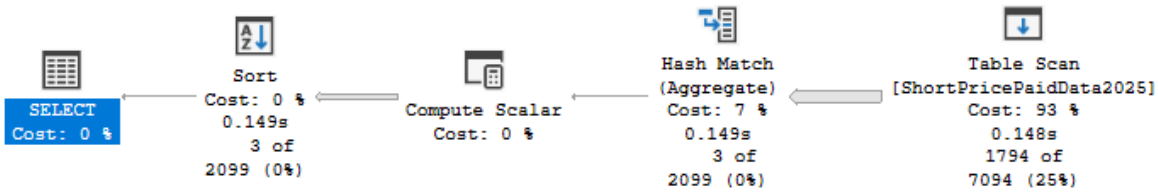


Figure 4: Execution plan without index

It is possible to see in this figure that in the case of the best index there is no sort which makes it very fast, while in the second index it has to do sort for the `GROUP BY`, and in the first index it does sort for the `GROUP BY` and it has to lookup the `locality` and `price` before joining to do the sort.

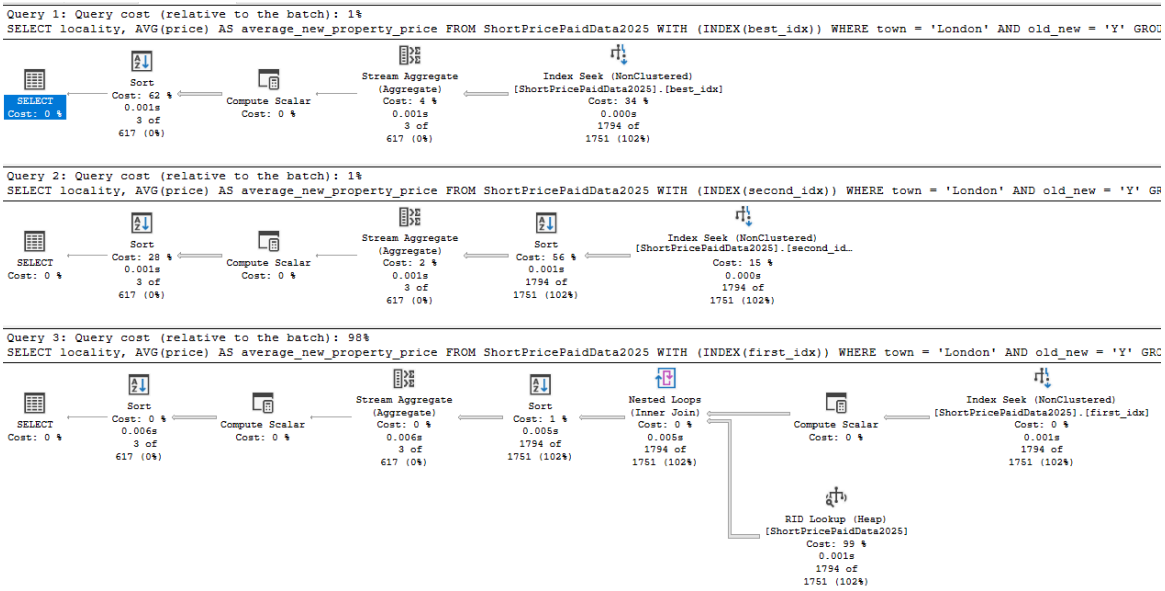


Figure 5: Execution plan with index

3.4 Cost Comparison

```
No Index: 11.515
First Index: 5.727
Second Index: 0.0806
Best Index: 0.0318
```

Listing 10: Retrieve estimated execution cost

3.5 Discussion

Initially, the query execution relied on a full table scan, which is inefficient for large datasets. This occurs because SQL Server had no efficient way to locate the relevant rows that satisfy the filtering conditions.

The introduction of a composite index on `(Old_New, Town)` allows SQL Server to directly locate rows matching the filtering requirements. Furthermore, by including `price` and `locality` in the index, we avoid an RID Lookup. Finally, by using `old_new` first in the index and using `locality` in the end of it, instead of just including it, we avoid using the sort caused by the `GROUP BY`.

A covering index is particularly efficient because all required columns are contained within the index itself. This eliminates the need for additional lookups to the base table, reducing both I/O operations and execution time.

The transition from a table scan to an index seek demonstrates a significant improvement in query efficiency, reflected in the reduction of the Estimated Subtree Cost.

4 Task 4: Join and Group By Analysis

4.1 Join And Group Algorithms

```sql
----------LOOP JOIN , HASH GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
    GROUP BY district
) p24
JOIN (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (LOOP JOIN, HASH GROUP, MAXDOP 1);
----------HASH JOIN , HASH GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
    GROUP BY district
) p24
JOIN (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (HASH JOIN, HASH GROUP, MAXDOP 1);
----------MERGE JOIN , HASH GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
    GROUP BY district
) p24
JOIN (
```

```sql
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (MERGE JOIN, HASH GROUP, MAXDOP 1);
----------LOOP JOIN , ORDER GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
    GROUP BY district
) p24
JOIN (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (LOOP JOIN, ORDER GROUP, MAXDOP 1);
----------HASH JOIN , ORDER GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
    GROUP BY district
) p24
JOIN (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (HASH JOIN, ORDER GROUP, MAXDOP 1);
----------MERGE JOIN , ORDER GROUP ---------------
SELECT p24.district,
       p24.avg_price AS avg_2024,
       p25.avg_price AS avg_2025
FROM (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2024
```

```sql
    GROUP BY district
) p24
JOIN (
    SELECT district, AVG(price) AS avg_price
    FROM ShortPricePaidData2025
    GROUP BY district
) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (MERGE JOIN, ORDER GROUP, MAXDOP 1);
```

Listing 11: Forcing different join algorithms

4.2 Execution Plans

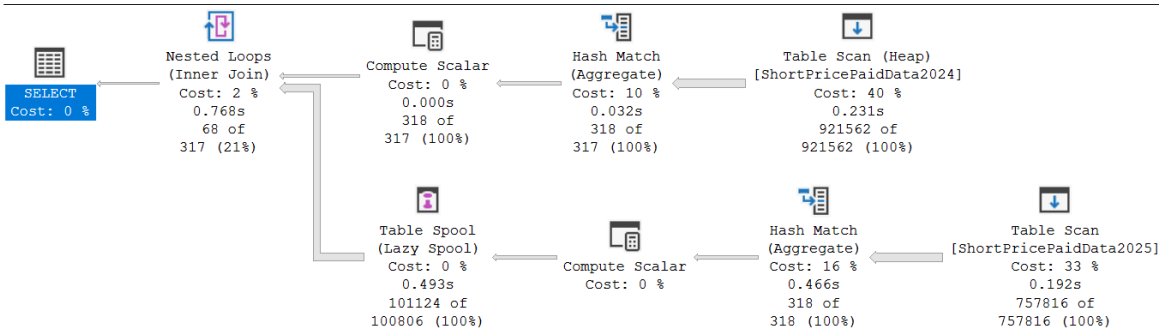


Figure 6: Loop Join with Hash Group

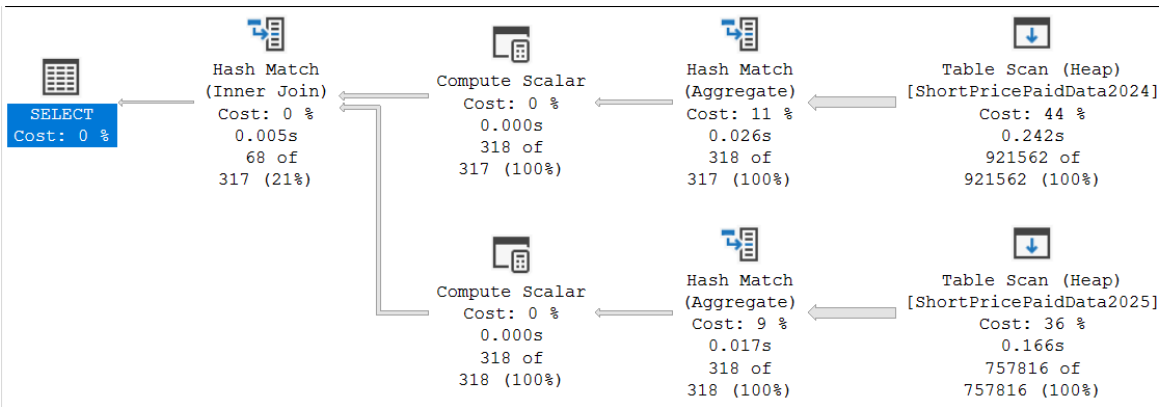


Figure 7: Hash Join with Hash Group

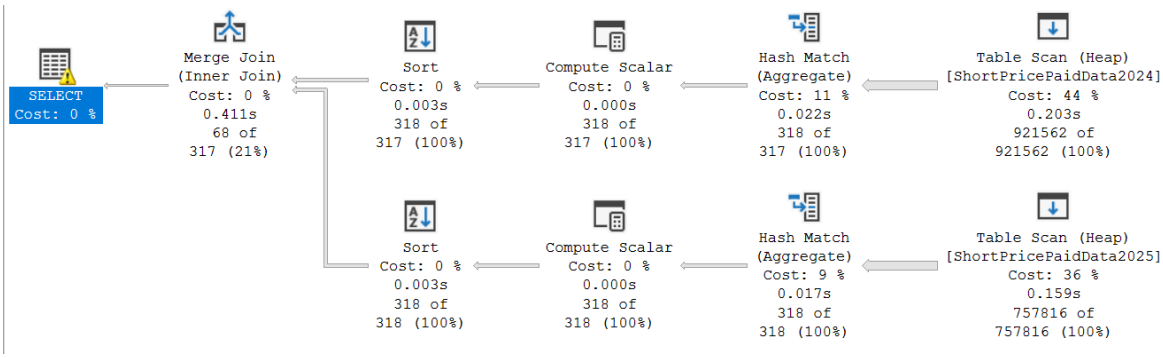


Figure 8: Merge Join with Hash Group

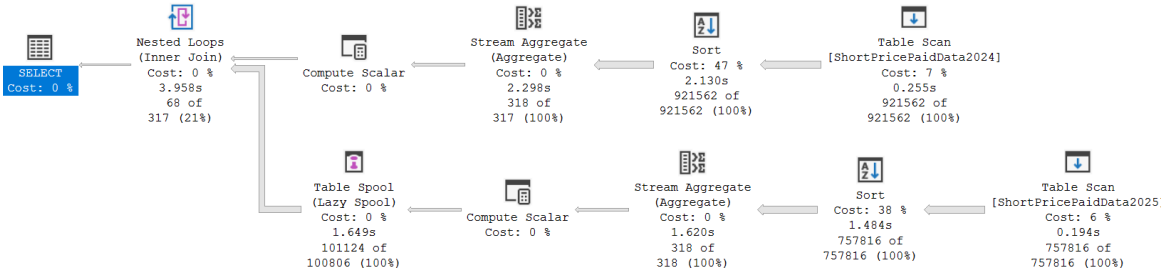


Figure 9: Loop Join with Order Group

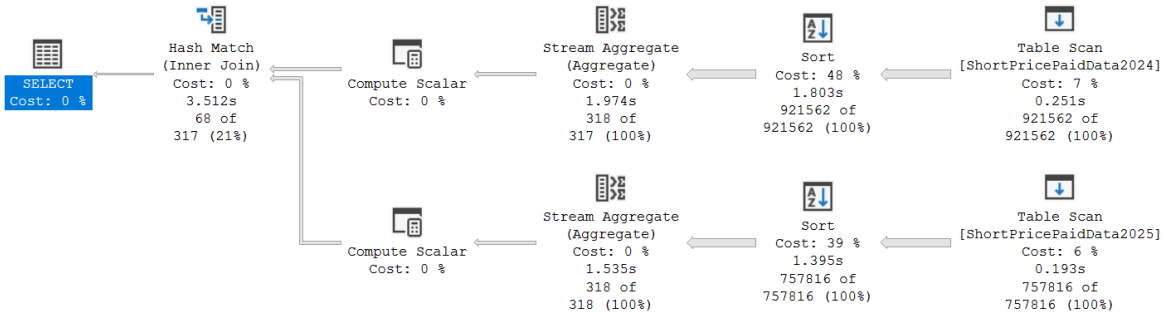


Figure 10: Hash Join with Order Group

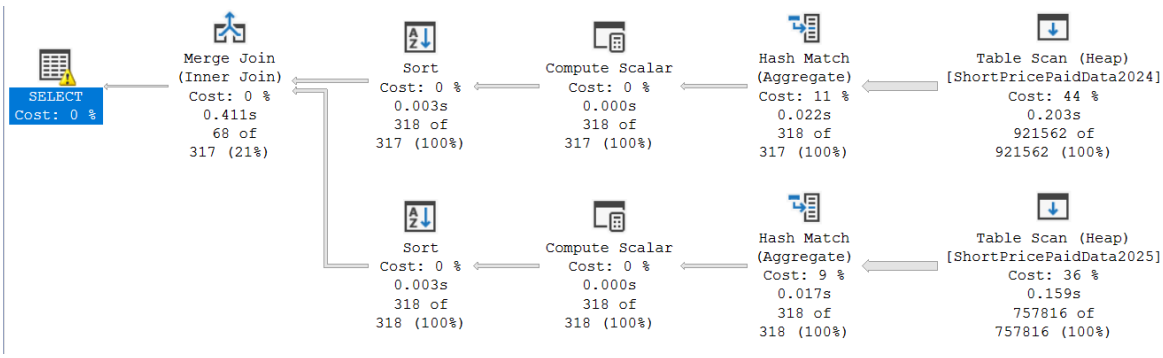


Figure 11: Merge Join with Order Group

4.3 Cost Comparison

```
Loop Join , Hash Group: 32.3489
Hash Join , Hash Group: 29.2655
Merge Join , Hash Group: 29.271
Loop Join , Order Group: 176.946
Hash Join , Order Group: 176.426
Merge Join , Order Group: 176.402
```

Listing 12: Retrieve estimated subtree cost

4.4 Discussion

In these results we can see a clear distinction between the two `Group` strategies. While the `Hash Group` has a cost between (29–32), the `Order Group` has a cost around (176). This happens because the data is not ordered by the attribute `district`, which means that the `Order Group` will need a sort operation before performing the aggregation, leading to a very high cost. In contrast, the `Hash Group` does not require the data to be ordered.

Regarding the `Join` algorithms, the cost differences are small when compared to the grouping strategies. Focusing on the `Hash Group` case, the `Hash Join` and `Merge Join` have very similar costs, while the `Loop Join` has a slightly higher cost.

The `Hash Join` is efficient because it matches rows using a hash table without requiring sorted data. The `Merge Join` processes sorted inputs sequentially, which requires both inputs to be ordered on the join key. In this case, SQL Server introduces a sort operator to guarantee this ordering, but since the intermediate result is small, the cost of this operation is negligible, which is why it appears as 0 in the execution plan. The `Loop Join`, on the other hand, is less efficient in this context because it performs repeated lookups for each row, making it less suitable for larger inputs.

5 Task 5: Query Tuning

5.1 Optimized Query

```sql
SELECT county, COUNT(*) AS properties_sold_both_years
FROM (
    SELECT DISTINCT p24.county, p24.paon, p24.street, p24.town
    FROM ShortPricePaidData2024 p24
    WHERE EXISTS (
        SELECT 1
        FROM ShortPricePaidData2025 p25
        WHERE p25.county = p24.county
          AND p25.paon = p24.paon
          AND p25.street = p24.street
          AND p25.town = p24.town
    )
) function
GROUP BY county
ORDER BY county
OPTION(MAXDOP 1);
```

Listing 13: Rewritten query

5.2 Execution Plans

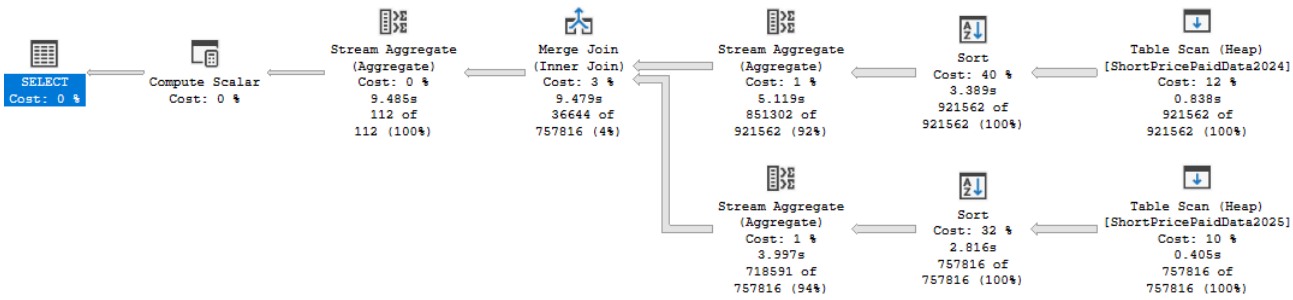


Figure 12: Original query: estimated subtree cost - 105.246

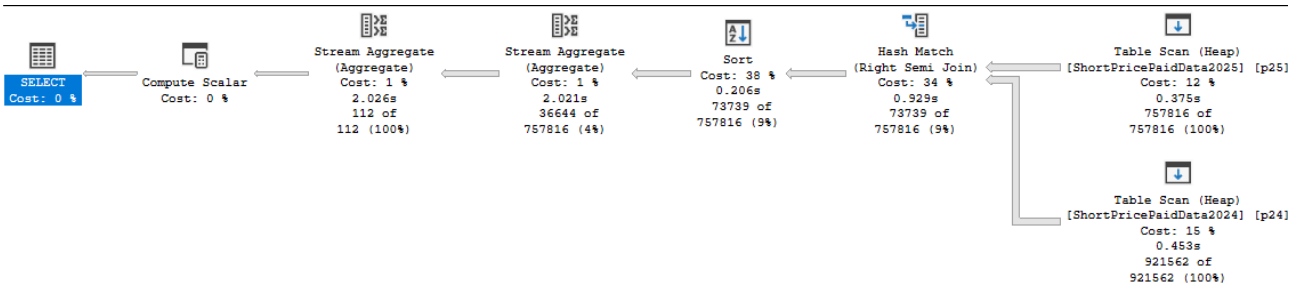


Figure 13: Optimized query: estimated subtree cost - 88.8174

5.3 Discussion

In the original query, duplicates are removed independently from both tables using `DISTINCT` before performing the join. This is expensive, as it requires processing the full datasets and realizing two intermediate results, which are then joined.

In the rewritten query, the join is replaced with the operator `EXISTS`, which performs a semi-join. Instead of combining both tables, it scans the 2024 table and checks whether a matching row exists in the 2025 table. This is more efficient because the search stops as soon as a match is found, avoiding unnecessary comparisons.

Additionally, the `DISTINCT` operation is applied only after filtering with `EXISTS`, meaning that duplicate elimination is performed on a smaller subset of rows. This reduces the cost of the aggregation compared to the original approach.

6 Task 6: Transactions and Isolation Levels

6.1 Isolation Level Uncommitted

At this stage we are dealing with the isolation level uncommitted and we can make a dirty read by starting the transaction and inserting a new row without committing on terminal 1.

```
------TERMINAL 1--------
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

BEGIN TRANSACTION;

INSERT INTO ShortPricePaidData2025
VALUES ('TEST', 9999999999, GETDATE(), 'XXXX', 'F', 'N', 'TEST', 'TEST
    ', 'TEST', 'TEST', 'TEST', 'TEST');
```

After this in terminal 2 we initiate a transaction and try to use the modified query from task 3.

```
------TERMINAL 2--------
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

BEGIN TRANSACTION

SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
GROUP BY locality
ORDER BY average_new_property_price DESC;
```

Which gives us this result.

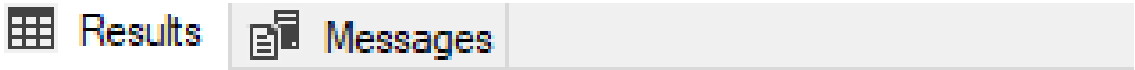
	locality	average_new_property_price
1	TEST	9999999999
2	KILMERSDON	100000000
3	DAVENTRY RAIL FREIGHT TERMINAL	37800000
4	REDHILL BUSINESS PARK	29000000
5	BIGGIN HILL AIRPORT	23500000
6	BLACKDOWN	18400000
7	WINCHEAP	14154785

Finally we will rollback in terminal 1, effectively making terminal 2 have read a value that doesn't exist in the database.

```
------TERMINAL 1--------
ROLLBACK;
```

6.2 Isolation Level Read Committed

At this stage we are dealing with the isolation level read committed where if we try the previous vulnerability, in the step where we execute the SELECT query we get a blank terminal 2 waiting for terminal 1 to finish committing.



However this level also has one vulnerability which is the unrepeatable read, to do this we will start by using the task 3 query in terminal 1.

```
------TERMINAL 1--------
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION

SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
GROUP BY locality
ORDER BY average_new_property_price DESC;
```

	locality	average_new_property_price
1	KILMERSDON	100000000
2	DAVENTRY RAIL FREIGHT TERMINAL	37800000
3	REDHILL BUSINESS PARK	29000000
4	BIGGIN HILL AIRPORT	23500000
5	BLACKDOWN	18400000
6	WINCHEAP	14154785
7	ANSTY PARK	13000000

Next we will insert a row into the table in terminal 2 and commit.

```
------TERMINAL 2--------
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION

INSERT INTO ShortPricePaidData2025
VALUES ('TEST2', 9999999999, GETDATE(), 'XXXX', 'F', 'N', 'TEST', '
   TEST', 'TEST', 'TEST', 'TEST', 'TEST');

COMMIT;
```

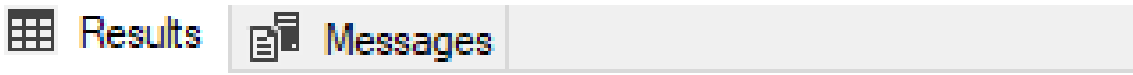
Finally we will do the task 3 query again, but this time it will not return the same value.

```
------TERMINAL 1--------
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
GROUP BY locality
ORDER BY average_new_property_price DESC;
```

	locality	average_new_property_price
1	TEST	9999999999
2	KILMERSDON	100000000
3	DAVENTRY RAIL FREIGHT TERMINAL	37800000
4	REDHILL BUSINESS PARK	29000000
5	BIGGIN HILL AIRPORT	23500000
6	BLACKDOWN	18400000
7	WINCHEAP	14154785

6.3 Isolation Level Serializable

In the final stage we are dealing with the isolation level serializable where if we try the previous vulnerability, in the step where we execute the insertion query we get a blank terminal 2 waiting for terminal 1 to finish committing the SELECT query.



In this level the transactions are completely protected, however the performance, consequentially, is going to drop drastically.

6.4 Discussion

Transaction isolation levels play a critical role in ensuring data consistency in concurrent environments.

At lower isolation levels, such as READ UNCOMMITTED, queries can read uncommitted data, leading to dirty reads. This can result in incorrect analytical outputs, especially in aggregation queries.

At higher isolation levels, such as SERIALIZABLE, SQL Server ensures that rows read during a transaction cannot be modified by other transactions until the first one completes. This prevents non-repeatable reads and guarantees consistent results.

However, higher isolation levels come at the cost of reduced concurrency, as they require more locking mechanisms. This trade-off between consistency and performance must be carefully considered when designing systems that support both analytical and transactional workloads.

7 Conclusion

This project allowed us to explore several important concepts related to database management and query optimization using SQL Server 2022.

Throughout the tasks, we worked with real-world data and implemented different techniques to improve performance. The creation of structured tables and the use of reduced datasets helped simplify data handling and made queries more efficient.

Partitioning showed how data can be organized to improve performance, especially when filtering by specific attributes. Indexing had a clear impact on query execution, allowing SQL Server to avoid full table scans and access only the necessary data.

The analysis of join and aggregation strategies highlighted that different execution plans can lead to different performance results, depending on the size and structure of the data.

Additionally, rewriting queries demonstrated that simpler and more direct approaches can reduce execution cost and improve efficiency.

Finally, the study of transaction isolation levels showed how concurrency can affect query results, and how choosing the appropriate isolation level is important to ensure data consistency.

8 Project Repository

All SQL scripts, queries, and supporting materials developed for this project are available in the following GitHub repository:

https://github.com/Afonso-Mousinho-Gomes/AOBD-project

8.1 Repository Structure

The repository is organized to reflect the different tasks of the project:

- **Task 1:** Scripts for database creation and data import
- **Task 2:** Partitioning implementation and verification queries
- **Task 3:** Index creation and query optimization experiments
- **Task 4:** Join and aggregation analysis queries
- **Task 5:** Query tuning and rewritten queries
- **Task 6:** Transaction experiments and isolation level demonstrations
- **Task 6 term 2:** Transaction experiments and isolation level demonstrations second terminal

Declaration

We declare that this project was developed by our group and is the result of our own work, except where explicitly stated.

We used external resources to support our understanding of certain concepts. In particular, we used ChatGPT to:

- Better understand how to perform `BULK INSERT` operations in SQL Server;
- Explore and clarify the use of SQL operators such as `NTILE` and `EXISTS`;
- Support general understanding of query optimization techniques.

All code, analysis, and conclusions presented in this report were implemented and validated by the authors.

References

[1] Microsoft, *SQL Server Documentation*, Available at: https://learn.microsoft.com/en-us/sql/

[2] Microsoft, *BULK INSERT (Transact-SQL)*, Available at: https://learn.microsoft.com/en-us/sql/t-sql/statements/bulk-insert-transact-sql

[3] Microsoft, *NTILE (Transact-SQL)*, Available at: https://learn.microsoft.com/en-us/sql/t-sql/functions/ntile-transact-sql

[4] Microsoft, *EXISTS (Transact-SQL)*, Available at: https://learn.microsoft.com/en-us/sql/t-sql/language-elements/exists-transact-sql

[5] OpenAI, *ChatGPT*, Available at: https://chat.openai.com